

提高级 CSP-S 第 5 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 表达式 $a \times (b+c) - d/f$ 转后缀表达式的结果是()。
A. $abc \times + df / -$ B. $abc + \times df / -$ C. $bc + a \times df / -$ D. $abc + \times - df /$
- 以下字符串中,字典序最小的是()。
A. NOIP2017 B. NOIPC C. NOIPC++ D. NOIPP
- 在 C++ 中,表达式 $('O' - 'I')^{13\%9+5}$ 的值是()。
A. 7 B. 8 C. 14 D. 15
- 在 8 个数中,找出这组数的最小值与最大值,最坏情况下最少需要比较()次。
A. 9 B. 10 C. 12 D. 13
- 在 TCP/IP 协议族中,最核心的网络协议是()。
A. UDP B. HTTP C. TCP D. IP
- 应用快速排序的分治思想可以实现一个求第 K 大数的程序。假定不考虑极端的最坏情况,理论上可以实现的最低的算法期望时间复杂度为()。
A. $O(n^2)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$
- 在解决计算机主机与外设之间速度不匹配时通常设置一个缓冲池,主要将计算机输出的数据依次写入该缓冲区,而外设从该缓冲区池中取出数据。该缓冲池应该是一个()结构。
A. 堆栈 B. 队列 C. 二叉树 D. 链表
- $(1E34)_{16} - (676)_8$ 的结果是()。
A. $(1110001111110)_2$ B. $(7276)_{10}$
C. $(1C6C)_{16}$ D. $(16166)_8$
- 一棵二叉树前序遍历为 ABDECFGH,后序遍历为 EDBGFHCA,以下不是可能的中序遍历的是()。
A. DEBAFGCH B. EDBAGFCH C. DBEAFGCH D. BEDAFGCH
- 一个栈的输入顺序为 1,2,3,4,5,下列序列中不可能是栈的输出序列的是()。
A. 54321 B. 24135 C. 32541 D. 13542
- Linux 是一个用 C 语言写成的开源电脑操作系统内核,有大量的操作系统是基于 Linux 内核创建的。以下操作系统使用的不是 Linux 内核的是()。
A. Android B. CentOS C. Windows D. Ubuntu
- 在下列关于计算机算法的说法中,正确的是()。
A. 对于一个问题,我们能通过优化算法,不断降低其算法复杂度
B. 判断一个算法的好坏,主要依据它在某台计算机上具体实现时的运行时间
C. 一个算法必须至少有一个输入
D. 算法复杂度理论中,NPC 问题(NP 完全问题)仍是一个未解之谜
- 以下关于二叉树性质中,正确的描述的个数有()。
A. 包含 n 个结点的二叉树的高度至少为 $\log_2 n$
B. 在任意一棵非空二叉树中,若叶子结点的个数为 n_0 ,度为 2 的结点数为 n_2 ,则 $n_0 = n_2 + 1$
C. 深度为 k 的二叉树至多有 2^k 个结点

D. 没有一棵二叉树的前序遍历序列与后序遍历序列相同

E. 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2(n+1) \rfloor$

A. 0

B. 1

C. 2

D. 3

14. 排序算法是稳定的,这句话的意思是关键码相同的记录排序前后相对位置不发生改变,以下排序算法不稳定的()。

A. 直接插入排序

B. 快速排序

C. 冒泡排序

D. 归并排序

15. 给定 m 种颜色和有 n 个点的手环,要求用这个 m 种颜色给这条手环染色。其中旋转和翻转能够互相得到的算同一种染色方案,如对于 3 个点的手环,ABC 的染色方法与 BCA (旋转)、ACB(翻转)是相同的。当 $m=2, n=2$ 时,一共有三种染色方案,分别为 AA、AB、BB。那么当 $m=5, n=4$ 时,一共有()种染色方案。

A. 625

B. 160

C. 60

D. 120

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确的填√,错误的填×;除特殊说明外,判断题每题 1.5 分,选择题每题 4 分,共计 40 分)

1.

```
01 #include<iostream>
02 using namespace std;
03
04 unsigned short tot;
05 void Hanoi(int n, char A, char B, char C) {
06     ++tot;
07     if (n==1) {
08         cout<<A<<"->"<<C<<' / ' ;
09         return;
10     }
11     Hanoi(n-1, A, C, B);
12     cout<<A<<"->"<<C<<' / ' ;
13     Hanoi(n-1, B, A, C);
14 }
15
16 int main() {
17     int n;
18     cin>>n;
19     Hanoi(n, 'A', 'B', 'C');
20     cout<<' \n' << tot <<' \n' ;
21     return 0;
22 }
```

● 判断题

(1)(1 分)将第 06 行移到 13 行和 14 行之间,输出结果不会变化。()

(2)当输入的 $n=2$ 时,输出的第一行为 A->B/B->C/A->C/。()

(3)当输入的 $n=3$ 时,输出的第二行为 8。()

(4)本程序的含义可以是:有三根柱子,第一根柱子从上到下依次套有编号分别为 $1 \sim n$

的圆环,现在每次可以移动某个柱子顶部的圆环到另一个柱子的顶部上,并且要求编号较大的圆环要始终不能在编号较小的上面,输出一种操作次数最少的方案以及对应的操作次数。()

●选择题

(5)当输入的 $n=3$ 时,输出的第一行的第 21 个字符是()。

- A. A B. B C. C D. /

(6)(5 分)当 $n=17$ 时,程序输出的第二行为()。

- A. -1 B. 131071 C. 131072 D. 65535

2.

```
01 #include<iostream>
02 #include<iomanip>
03 #include<cstring>
04 using namespace std;
05 const int N=105;
06 int a[N][N];
07 int main() {
08     int n,x,y,count;
09     cin>>n;
10     memset(a,0,sizeof(a));
11     count=a[x=0][y=n-1]=1;
12     while (count<n*n) {
13         while (x+1<n && !a[x+1][y]) a[++x][y]=++count;
14         while (y-1>=0 && !a[x][y-1]) a[x][--y]=++count;
15         while (x-1>=0 && !a[x-1][y]) a[--x][y]=++count;
16         while (y+1<n && !a[x][y+1]) a[x][++y]=++count;
17     }
18     for (x=0;x<n;x++) {
19         for (y=0;y<n;y++) {
20             cout<<setw(5)<<a[x][y];
21         }
22         cout<<endl;
23     }
24     return 0;
25 }
```

●判断题

(1)(1 分)删除第 10 行,不影响程序运行结果。()

(2)将第 12 行改为“while (count<=n * n) {”,不影响程序运行结果。()

(3)当输入的 $n=4$ 时,程序输出的 $a[3][2]$ 的值为 15。()

●选择题

(4)(3 分)本题的时间复杂度为()。

- A. $O(n)$ B. $O(n^2)$ C. $O(n^3)$ D. $O(n^2 \log n)$

(5) 当输入的 $n=100$ 时, $a[33][66]$ 的值为()。

A. 8779

B. 8707

C. 10957

D. 8845

3.

```

01 #include<iostream>
02 #include<iomanip>
03 #include<cstring>
04 using namespace std;
05 string s;
06 int main() {
07     int k;    //限制输入的 0<=k<26
08     cin>>k>>s;
09     int n=s.length( );
10     for (int i=0;i<n;i++) {
11         if (s[i]<=' Z' && s[i]+k>' Z')
12             s[i]=(s[i]+k)%' Z'+' A'-1;
13         else if (' A'<=s[i] && s[i]<=' Z')
14             s[i]+=k;
15     }
16     char pre;
17     int st=-1;
18     for (int i=0;i<n;i++) {
19         if (s[i]<' A' || s[i]>' Z') {
20             if (st== -1) {
21                 st=i;
22                 pre=s[i];
23             } else {
24                 char tmp=s[i];
25                 s[i]=pre;
26                 pre=tmp;
27             }
28         }
29     }
30     if (st != -1)
31         s[st]=pre;
32     cout<<s<<endl;
33     return 0;
34 }
```

● 判断题

(1) (1分) 删除第 30 行和第 31 行, 不影响程序运行结果。()

(2) 如果输入的 s 不含大写字母, 则输出结果与 k 的值无关。()

(3) 如果知道输出结果, 能够反推出唯一的输入结果。()

(4) 当 k 的值确定时, 不存在两个不同的输入使得输出相同。()

● 选择题

(5) 如果输入是 6 KU96APY5, 则输出为()。

A. QB96GWE5

B. QA59GVE6

C. PA59GWF6

D. PB96GWE5

(6) (5 分) 如果输出是 ab1287F2Tguz, 则输入可能为()。

A. 0 ab1287F2Tguz

B. 3 b1287BgTuza

C. 16 b12872PgDuza

D. 13 b12872MgAuza

三、完善程序 (单选题, 每小题 3 分, 共 30 分)

1. (拓扑排序) 在我们所学的课程中, 部分课程之间可能存在依赖关系, 如我们在学习图论知识之前, 需要先学习离散数学中的基础知识。一门课可能有若干先修课程。现在李老师需要安排一些课程的授课计划, 排课需要遵从一定的规则, 即只有修习完某课程的全部课程后, 才能修习该课程。

在本例中, 用 $1 \sim n$ 表示 n 个课程, 用 $x y$ 表示 x 是 y 的先修课程。输入数据保证图中没有环与重边。要求输出任意一个可行的授课顺序。图用邻接矩阵方法储存。

```
01 #include<iostream>
02 #include<cstring>
03 #include<queue>
04 using namespace std;
05 const int N=105;
06 int a[N][N];
07 int in[N], s[N];
08 int n, m, u, v;
09 void Topo() {
10     queue<int> q;
11     int cnt;
12     for (int i=1; i<=n; i++)
13         if ( ① )
14             q.push(i);
15     while (!q.empty()) {
16         int cur=q.front();
17         q.pop();
18         s[cnt++] = ② ;
19         for (int i=1; i<=n; i++) {
20             if ( ③ ) {
21                 ④ ;
22                 if (in[i]==0)
23                     q.push(i);
24             }
25         }
26     }
```

```

27 }
28 int main() {
29     memset(in,0,sizeof(in));
30     memset(a,0,sizeof(a));
31     cin>>n>>m;
32     for (int i=0;i<m;i++) {
33         cin>>u>>v;
34         in[v]++;
35         ⑤ ;
36     }
37     Topo();
38     for (int i=0;i<n;i++) {
39         if (i)
40             cout<<' ';
41         cout<<s[i];
42     }
43     cout<<endl;
44     return 0;
45 }

```

(1)①处应填()。

A. in[i] == 0 B. in[i] == 1 C. a[1][u] == 0 D. a[1][u] == 1

(2)②处应填()。

A. q.front() B. cur C. q.back() D. s[cur]

(3)③处应填()。

A. !--in[i] B. a[u][v] == 1 C. a[cur][i] == 1 D. !in[i]

(4)④处应填()。

A. in[i]++ B. in[i]-- C. q.pop() D. q.push(cur)

(5)⑤处应填()。

A. in[u]++ B. in[u]-- C. a[u][v] = 1 D. a[v][u] = 1

2. (8-皇后问题)要求用回溯法求解8-皇后问题,使放置在 8×8 棋盘上的8个皇后彼此不受攻击,即:任何两个皇后都不在同一行、同一列或同一斜线上。请输出8皇后问题的所有可行解。

```

01 #include<iostream>
02 #include<cmath>
03 using namespace std;
04 bool Place(int k,int i,int*x) {
05     for (int j=0;j<k;j++)
06         if ((x[j]==i) || ① )
07             return false;
08     return true;
09 }

```

```

10
11 void NQueens(int k,int n,int*x) {
12     for (int i=0;i<n;i++) {
13         if (Place(k,i,x)) {
14             ②;
15             if ( ③ ) {
16                 for (int j=0;j<n;j++) cout<<x[j]<<" ";
17                 cout<<endl;
18             }
19             else {
20                 ④;
21             }
22         }
23     }
24 }
25 int main() {
26     int x[8];
27     for (int i=0;i<8;i++) x[i]=-1;
28     ⑤;
29     return 0;
30 }

```

(1) ①处应填()。

A. $x[j] == i-1$

B. $\text{abs}(x[j]-i) \leq 1$

C. $\text{abs}(x[j]-k) == i-j$

D. $\text{abs}(x[j]-i) == k-j$

(2) ②处应填()。

A. $x[i] = k$

B. $x[k] = n-i$

C. $x[k] = i$

D. $x[i] = x[k]$

(3) ③处应填()。

A. $k > n-1$

B. $k \geq n-1$

C. $k == n$

D. $k > n$

(4) ④处应填()。

A. $\text{NQueens}(k, n-1, x)$

B. $\text{NQueens}(k+1, n, x)$

C. $\text{NQueens}(k+1, n-1, x)$

D. $\text{cout} << x[k] << " "$

(5) ⑤处应填()。

A. $\text{NQueens}(1, 8, x)$

B. $\text{NQueens}(0, 7, x+1)$

C. $\text{NQueens}(0, 8, x)$

D. $\text{NQueens}(1, 8, x+1)$